

U2T1 Preguntas G4

Integrantes:

- Lasso Sebastián
- Valencia Yuliana
- Villarroel Justin

1. En el CRUD del Estudiante, ¿por qué Composite permite tratar de forma uniforme a un Estudiante individual y a un GrupoEstudiantes? Explique qué ventaja aporta esta decisión al controlador y qué riesgo aparece si se permiten agrupaciones sin reglas.

Porque ambos comparten una misma interfaz, la cual nos permite tratar a la hoja que es el estudiante como también al compuesto que sería el grupo de estudiantes, puedan tener el mismo método sin que se base en clases padres e hijos, sino como una implementación de polimorfismo.

Una ventaja sería que en el controlador se evita tener mayor cantidad de condiciones, porque la interfaz se encarga de ejecutar la acción y espera a que cada objeto resuelva la petición asignada.

Un riesgo sería si existiesen compuestos que se contengan a sí mismos o a otros grupos, en ese caso podría causar bucles infinitos y podría llegar a colapsar la memoria de la aplicación.

2. Compare Proxy y Bridge en el mismo CRUD: ¿cuál controla el acceso a las operaciones y cuál separa la lógica de negocio de la implementación de persistencia? Justifique con ejemplos de clases del documento.

El patrón encargado de controlar el acceso es el Proxy, el cual se evidencia en la clase ProxyProxyEstudiante al interceptar las llamadas a IServicioEstudiante para validar permisos y registrar logs mediante sus métodos privados antes de delegar la petición al servicio real. Por otro lado, el patrón que separa la lógica de negocio de la implementación de persistencia es el Bridge, el cual utiliza la clase abstracta AbstractEstudiante en el paquete de servicios para conectarse mediante composición con la interfaz IImplementadorMemoria (a través del atributo # implementador), logrando que las reglas del negocio queden totalmente desacopladas de los detalles técnicos del almacenamiento en ImplementadorListaMemoria.

3. Si el sistema evoluciona de una lista en memoria a una base de datos real y además se agregan roles Admin/Invitado, ¿cómo deberían colaborar Composite, Proxy y Bridge sin mezclar responsabilidades? Argumente qué clase o interfaz debería cambiar y cuál debería mantenerse estable.

Para evolucionar el sistema sin mezclar responsabilidades, el Proxy (ProxyProxyEstudiante) debe modificarse para recibir el rol del usuario ("Admin" o "Invitado") en su constructor y validar activamente en su método verificarAcceso() que sólo los administradores ejecuten cambios, bloqueando a los invitados. Por su parte, el Bridge permite cambiar la persistencia creando una nueva clase concreta (como ImplementadorMySQL) que implemente la interfaz existente IImplementadorMemoria, logrando que la base de datos real se conecte sin alterar el resto del sistema. Mientras tanto, el Composite (IComponenteAcademico, Estudiante y GrupoEstudiantes) y el controlador (ControlEstudiante) se mantienen completamente estables e intactos, ya que a la estructura jerárquica de los alumnos y a la lógica de control no les compete saber quién maneja los permisos de acceso ni en qué tecnología física se están almacenando los datos.